BSc (Hons) in **Computer Science**

Faculty of Computing

SE3062 : Intelligent Systems

Year 3 · Semester 1 · 2026 Semester 2

Practical 02

> **Objective & Lecture Mapping:** In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

> To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: `IF food_here THEN suck; IF wall_ahead THEN turn_left; ELSE move_forward`

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
   Example Logic: `IF wall_ahead AND left_is_visited THEN turn_right`
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

> Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

A table-driven agent works by storing one row for every possible percept sequence it could ever encounter, with the correct action pre-computed for each. This is impossible for Chess (and any complex environment) because the table would need an entry for every distinct sequence of board-states-so-far, and the number of legal Chess positions alone is astronomically large (commonly cited around 10^43–10^47), before even accounting for sequences of positions rather than single snapshots. No physical storage medium could hold a table that size, and no one could hand-author it.

As the agent's lifetime increases, this gets categorically worse, not just bigger: the table's size grows with the number of possible percept sequences, which grows exponentially with each additional time step (since every new percept can follow any prior sequence). A table sized for a short game becomes irrelevant the moment the game runs longer than anticipated — there's no bounded table that covers unbounded or even very long lifetimes. This is the core argument for why real agents need a program that generates actions from percepts algorithmically, rather than a lookup structure that merely stores them.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent`. Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

The classic condition-action rule pattern looks like:

python
```
def sense_and_act(self, percept: dict) -> str:
    if percept['wall_ahead']:
        return random.choice(['Left', 'Right', 'Down', 'Up'])
    if percept['food_here']:
        return 'Stay'  # or whatever "collect" maps to
    return random.choice(self.actions_pool)
```

The if percept[...]: return ... lines are the condition-action rules — each is a direct mapping "if condition X currently holds in the percept, take action Y," with no memory of anything before this single percept. The defining trait per Lecture 02 is that the rule fires purely off the current percept dictionary; there's no state variable being read or updated, no history consulted — the entire decision is f(current_percept) → action. That's what makes it a simple reflex agent rather than model-based: the rule set is fixed and stateless, so the same percept always produces the same rule match (though not necessarily the same action, if randomness is involved within a branch — the fact that test_bfs... sorry, test_simple_reflex_logic only checks that an action came back, not a fixed one, reflects this).

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

The failure mode: feed the agent the same percept {'wall_ahead': True, 'food_here': False} twice in a row (this is exactly what test_model_based_memory does), and a simple reflex agent's condition-action rule for "wall ahead" fires identically both times — e.g., always picking 'Up' when a wall blocks it, or picking randomly but with no memory of what it already tried. If the environment is only partially observable (the percept doesn't tell the agent which specific wall or direction it's blocked in beyond a single boolean, and doesn't reveal enough of the surrounding layout to know a bounce-back is happening), the agent can't distinguish "first time hitting this wall" from "third time hitting this exact same wall from the same position." Because there's no percept history — no record of past states or actions — the agent has no way to detect "I have already tried this action and it failed," so it cannot break the cycle by reasoning "avoid repeating." The combination is what causes the loop: partial observability hides the fact that the agent is revisiting the same state, and the lack of history means even if it could infer that, it has nowhere to store the inference. The agent is condemned to re-derive the same (bad) action from the same (uninformative) percept, forever.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent`. Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

Evaluating this against the two theoretical components:

Transition Model (how the world evolves given an action) — here it's implicit and weak: the agent doesn't actually simulate "if I move Up, where will I end up and will there be a wall there?" It only remembers what it last tried, not a predictive model of consequences. A stronger implementation would maintain a belief grid or explicit function predict(state, action) -> new_state, letting the agent reason forward rather than just avoid repetition. As written, the transition model is barely present — it's really just "don't repeat the last failed action," which sidesteps modeling the world's dynamics rather than representing them.
Sensor Model (how the agent's percepts relate to the true state / how actions affect what will be sensed next) — similarly thin: the agent doesn't reconcile wall_ahead readings with a persistent internal map of where walls are, so it can't predict future percepts either. It reacts to the current boolean and stores only the bare minimum (last_action) needed to break the immediate loop.

Finalize and Lock Lab 02 Documentation